

 **MASTERCLASS**

RAG vs Vectorless RAG

Similarity search ruled retrieval for years - until reasoning showed up



Al Coach John



FOLLOW

 **Repost**

⚠ THE PROBLEM

The Problem With Traditional RAG Nobody Talks About

Your RAG system retrieves by similarity. But what you need is relevance. Those are not the same thing.

⚡ Real Scenario



A financial analyst asks: "What was the net operating loss carryforward mentioned in the risk factors section of the 2023 annual report?"

🔍 Vector RAG Process

- Embeds query, finds semantically similar chunks
- Returns chunks about "losses" from random pages

🎯 The Reality

- Actual answer is in Note 12 on page 87
- Vector RAG has no idea Note 12 even exists as a logical unit



This distinction is survivable for simple QA. It becomes **catastrophic** for financial reports, legal contracts, and technical manuals.



AI Coach John



FOLLOW

↻ Repost

How Traditional Vector RAG Works

```
PDF / Docs
↓
[Text Extraction]
↓
[Chunking] → 512-1024 token chunks
↓
[Embedding Model] → dense vector
↓
[Vector Database] → Pinecone / Chroma
↓
User Query → cosine similarity search
↓
LLM generates answer
```

✖ Where it breaks:

- ✂ **Chunking cuts across logical boundaries:** a table gets split, a clause loses its context.
- 🔗 **Cross-reference blindness:** page 12 says "see Appendix B". Vector RAG has no mechanism to follow that reference.
- 🧠 **Limited reasoning:** retrieval is based on one-shot similarity search and cannot explore structure step by step.

ACCURACY ON COMPLEX DOCS STAYS < 60%



Al Coach John



FOLLOW

↺ Repost

What is **Vectorless RAG** (PageIndex)?

A completely new pipeline that replaces vector databases and chunking with **LLM reasoning** and a **hierarchical tree index**.

- ✓ Instead of splitting documents artificially, it builds a smart "Table of Contents" tree where each node represents a natural section.
- 📖 The core insight borrowed from AlphaGo: Don't search exhaustively - use a learned strategy to navigate intelligently.

📖 THINK OF IT LIKE THIS:

When you need to find the refund clause in a 200-page legal contract, you don't read every page.

You go to the table of contents → find "Payment Terms" → navigate to "Refund Policy" → read that section.

PageIndex forces the LLM to behave exactly the same way.

🔄 Open-sourced by VectifyAI

github.com/VectifyAI/PageIndex



How Vectorless RAG Works

Two-step pipeline - index once, reason on every query.

STEP 1: INDEX

PDF

↓
[LLM reads full doc]

↓
[Builds Tree Index]
Document

├ Chapter 1
│ └ 1.1 Background
├ Chapter 2
│ └ 2.1 Study Design

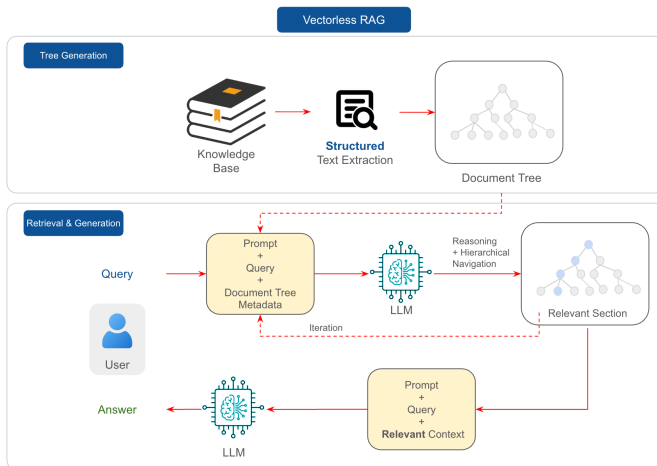
Each node has: title, summary, page range

STEP 2: RETRIEVE

User Query

↓
[LLM reads the tree]
↓
[Reasons: "Answer in 2.1"]
↓
[Fetches only those pages]
↓
[LLM generates answer]

Generates answer with exact page citations



- ★ **No vector DB:** uses document structure and LLM reasoning.
- No chunking:** organized into natural sections.



AI Coach John



FOLLOW

↻ Repost

Head to Head

Query: "What are the key risk factors related to foreign currency exposure?" in a 300-page annual report.

✗ VECTOR RAG

- Retrieves chunks with words like "currency", "foreign"
- Returns 5 scattered chunks, misses cross-reference to Note 8

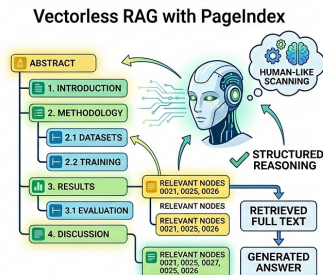
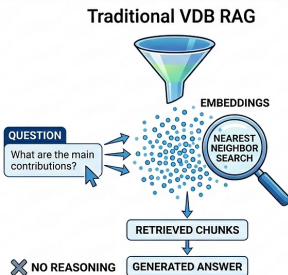
ANSWER: INCOMPLETE

✓ VECTORLESS RAG

- LLM reads tree → identifies "Risk Factors → Market Risk" node
- Fetches section + follows internal reference to Note 8

ANSWER: COMPLETE

VECTORLESS RAG: HUMAN-LIKE DOCUMENT NAVIGATION



📈 BENCHMARK RESULT (FINANCEBENCH)

~50%

Vector RAG

98.7%

Vectorless RAG



Al Coach John



FOLLOW

↻ Repost

Where **Vector RAG** Still Wins

Vectorless RAG is not a universal upgrade.

Millions of Products

E-commerce search across 2M descriptions. No hierarchy, flat texts.

Scales horizontally

Wikipedia Dumps

General knowledge QA over mixed text soup. No global hierarchy.

Similarity excels

Sub-100ms Real-Time Search

Vector RAG is a single fast lookup. Vectorless requires sequential LLM inference calls to navigate the tree, which is slower.

Speed & Cost at scale

THE RULE

Vector-based retrieval works best for large collections of unrelated or loosely structured documents where semantic similarity is sufficient.



AI Coach John



FOLLOW

 **Repost**

Where **Vectorless RAG** Wins

The documents where PageIndex changes everything:

Financial Reports

10-K filings, earnings docs.
Numbered sections, specific clause locations.

Legal Contracts

Clauses reference other clauses ("subject to Section 14.3").
PageIndex navigates directly.

Technical Manuals

"See Chapter 7 for details". Cross-references are the answer.

Academic Papers

Structured by section, numbered figures, tables, referenced appendices.

“ MICROSOFT'S ASSESSMENT

According to a March 2026 technical blog, traditional RAG limitations become **"catastrophic"** in financial documents, legal contracts, and technical manuals where structural understanding is critical.



Al Coach John



FOLLOW

 Repost

Limitations of Vectorless RAG

Know the tradeoffs before you switch:

- ⌚ **Latency:** Multiple LLM calls to navigate the tree vs one vector search.
- \$ **Cost:** Reasoning over tree structure consumes more tokens per query.
- 📁 **Document Scoped:** Currently scopes to single documents well. Multi-document corpora still favor vector RAG.
- 🔄 **Index Rebuild Cost:** Changing the document means rebuilding the tree via LLM calls.

🔗 BOTTOM LINE

- ↔ PageIndex trades infrastructure complexity and retrieval accuracy for higher per-query latency and cost.
- ✓ For structured documents where correctness is non-negotiable - that tradeoff is worth it.



AI Coach John



FOLLOW

↻ Repost

How to Implement Vectorless RAG

```
pip install pageindex
```

Step 1 - Build the tree index:

```
from pageindex import PageIndex

pi = PageIndex(api_key="your-key")
# LLM builds the hierarchical tree
index = pi.index_document("annual_report.pdf")
```

Step 2 - Query with reasoning:

```
# LLM navigates tree → fetches pages → answers
result = pi.query(
    index=index,
    question="Key risks for foreign currency?"
)

print(result.cited_pages) # Exact pages: [42, 87]
```



Hosted Version

chat.pageindex.ai



Agent Integration

API + MCP access



AI Coach John



FOLLOW

Repost

Decision Framework

Answer these 3 questions:

Q1: IS DOCUMENT LOGICALLY STRUCTURED?

- Yes (reports, manuals) → PageIndex
- No (social media, mixed corpus) → Vector RAG

Q2: HOW MANY DOCUMENTS?

- Single or few long docs → PageIndex
- Thousands of short docs → Vector RAG

Q3: CORRECTNESS VS SPEED?

- Correctness critical (finance, legal) → PageIndex
- Speed critical, approx OK → Vector RAG



AI Coach John



FOLLOW

 Repost

Cheatsheet

Dimension	Vector RAG	Vectorless RAG
Method	Cosine similarity	LLM tree reasoning
Chunking	Required	None
Vector DB	Required	None
Cross-ref	Blind	Follows them
Best for	Unstructured corpus	Structured docs
Accuracy	~50%	98.7%
Latency	Fast	Slower
Scale	Millions of docs	Single / Few



THE RULE



Similarity $\neq$ Relevance



When structure carries meaning - reasoning wins



When scale and fuzziness dominate - similarity wins



Al Coach John



FOLLOW

↺ Repost